Verilog

```
module traffic_controller(
    input CLK,
    input GO_GREEN,
    input GO_YELLOW,
    input GO_RED,
    output reg LIGHT_GREEN,
    output reg LIGHT_YELLOW,
    output reg LIGHT_RED
);
```

Port list (signals that connect this module to the outside world)

```
// state encoding
parameter GREEN = 2'b00,
          YELLOW = 2'b01,
          RED = 2'b10;
```

No. of bits 2

binary

[note: 3 states = 2 bits required]

```
reg [1:0] current_state, next_state;

always @(posedge CLK)
begin
    current <= next_state;
end
```

[means, All state changes happen only when clock rises. Ipours code

FSM changes state only when clock goes from 0 → 1.]

```
// Next state logic
always @(*)
begin
    case (current_state)
        GREEN:
            if (GO_YELLOW)
                next_state = YELLOW;
            else
                next_state = GREEN;
        YELLOW:
            if (GO_RED)
                next_state = RED;
            else
                next_state = YELLOW;
```

[without clock state immediately changes

always @(*)

mathematically Next_state = f(current state, inputs)

pure combinational logic

No memory "like".

means Run this block whenever any signal inside changes.]



```
        RED:
            if(GO_GREEN)
                next_state = GREEN;
            else
                next_state = RED;
        default:
            next_state = RED;
    endcase
end

// output logic (Moore FSM)
always @(*)
begin
    case (current_state)
        GREEN:
        begin
            LIGHT_GREEN = 1'b1;
            LIGHT_YELLOW = 1'b0;
            LIGHT_RED = 1'b0;
        end
        YELLOW:
        begin
            LIGHT_GREEN = 1'b0;
            LIGHT_YELLOW = 1'b1;
            LIGHT_RED = 1'b0;
        end
        RED:
        begin
            LIGHT_GREEN = 1'b0;
            LIGHT_YELLOW = 1'b0;
            LIGHT_RED = 1'b1;
        default:
        begin
            LIGHT_GREEN = 1'b0;
            LIGHT_YELLOW = 1'b0;
            LIGHT_RED = 1'b1;
        end
    endcase
end
endmodule
```

[state = GREEN → LIGHT_GREEN = 1
LIGHT_GREEN is just a signal
that tells outside world: "Green
light should glow"]

Testbench code

```
`timescale 1ns/1ps
module traffic_controller_
    // Testbench signals
    reg CLK;
    reg GO_GREEN;
    reg GO_YELLOW;
    reg GO_RED;

    wire LIGHT_GREEN
    wire LIGHT_YELLO
    wire LIGHT_RED;
// Instantiate the Design
traffic_controller DU
        .CLK(CLK)
        .GO_GREEN(
        .GO_YELLOW(
        .GO_RED(G
        .LIGHT_GREE
        .LIGHT_YELL
        .LIGHT_RED
    );

// clock generation (10ns p
always #5 CLK = ~CL
// TEST sequence
initial
begin
    // Initialize
    CLK = 0;
    GO_GREEN = 0;
    GO_YELLOW = 0;
    GO_RED = 0;

    // wait some time
    #10;
// Move from RED to GREEN
    GO_GREEN = 1;
    #10;
    GO_GREEN = 0;
```

Test bench code

```
`timescale 1ns/1ps

module traffic_controller_tb;

    // Testbench signals
    reg CLK;
    reg GO_GREEN;
    reg GO_YELLOW;
    reg GO_RED;

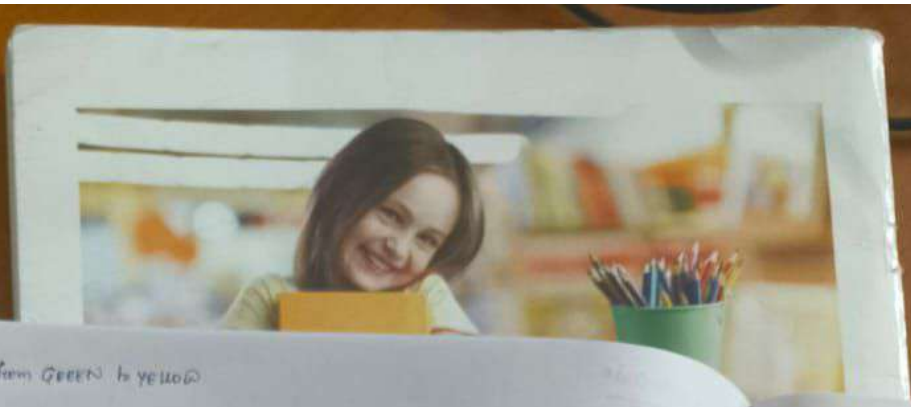
    wire LIGHT_GREEN;
    wire LIGHT_YELLOW;
    wire LIGHT_RED;

// Instantiate the Design Under Test (DUT)
traffic_controller DUT(
        .CLK(CLK)
        .GO_GREEN(GO_GREEN),
        .GO_YELLOW(GO_YELLOW),
        .GO_RED(GO_RED),
        .LIGHT_GREEN(LIGHT_GREEN),
        .LIGHT_YELLOW(LIGHT_YELLOW),
        .LIGHT_RED(LIGHT_RED)
    );

// clock generation (10ns period)
always #5 CLK = ~CLK;

// TEST sequence
initial
begin
    // Initialize
    CLK=0;
    GO_GREEN=0;
    GO_YELLOW=0;
    GO_RED=0;

  // wait some time
    #10;
// Move from RED to GREEN
    GO_GREEN=1;
      #10;
    GO_GREEN=0;
```



```
// Move from GREEN to YELLOW
#20;
GO_YELLOW = 1;
#10;
GO_YELLOW = 0;
// Move from YELLOW to RED
#20;
GO_RED = 1;
#10;
GO_RED = 0;
#50;
$stop; // Stop simulation
end
end module.
```

CTL = Computation Tree Logic

⇒ Safety property

Safety means: "something bad never happens"

In traffic light

two lights ON at same time (Bad condition)

If your outputs are LIGHT_GREEN, LIGHT_YELLOW, LIGHT_RED

CTL formula:

AG!(LIGHT_GREEN & LIGHT_RED)

AG!(LIGHT_GREEN & LIGHT_YELLOW)

AG!(LIGHT_RED & LIGHT_YELLOW)

meaning for all paths, always NO two lights are ON together.

⇒ Liveness property ⇒ Something eventually happens.

example: If system is RED, it should eventually go to GREEN

AG(RED → AF GREEN)

AG(GREEN → AF YELLOW)

AG(YELLOW → AF RED)

Meaning: The cycle continues forever

Functional property (Correct T

Example: If we are in GREEN

CTL Formula:

AG((GREEN & GO_YE

AG((RED & GO_GR

AG((YELLOW & GO_R

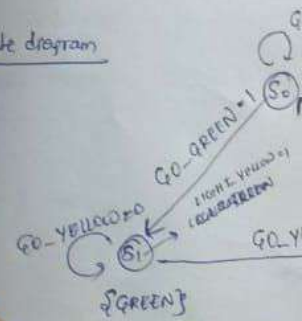
Perform model checking by label

S_0 = RED

S_1 = GREEN

S_2 = YELLOW

State diagram



by labeling

AF GREEN

$AG(RED \rightarrow AF\ GREEN)$ ⇒ For all paths, if state is RED, then eventually GREEN will occur.

$AG(GREEN \rightarrow AF\ YELLOW)$

$AG(YELLOW \rightarrow AF\ RED)$

Meaning: The cycle continues forever.

Functional property (Correct Transition)

Example: If we are in GREEN and GO_YELLOW = 1, next state must be YELLOW.

CTL formula:

$AG((GREEN \wedge GO_YELLOW) \rightarrow AX\ YELLOW)$

$AG((RED \wedge GO_GREEN) \rightarrow AX\ GREEN)$

$AG((YELLOW \wedge GO_RED) \rightarrow AX\ RED)$

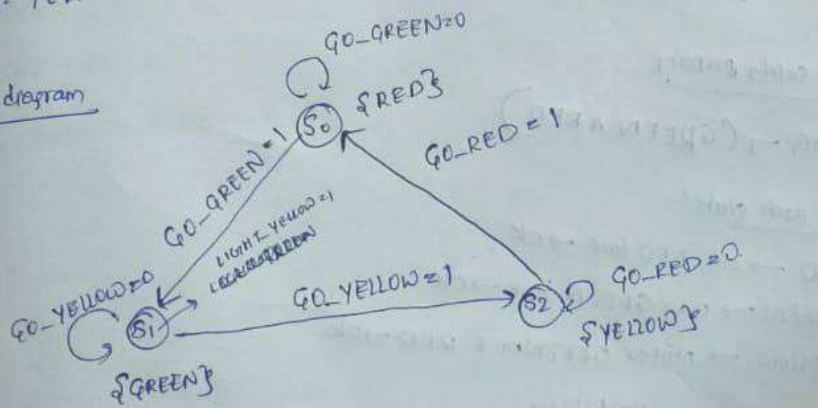
Perform model checking by labeling.

$S_0 = RED$
$S_1 = GREEN$
$S_2 = YELLOW$

Transitions:

RED —(GO_GREEN)→ GREEN
GREEN —(GO_YELLOW)→ YELLOW
YELLOW —(GO_RED)→ RED

State diagram



Model checking by labeling

FIRST Evaluate AFG_{GREEN}

$AF\phi$ means ⇒ on all paths, eventually ϕ holds.

So we find states from which GREEN will eventually be reached.

State GREEN ⇒ already Reached.

State YELLOW ⇒ yellow → Red → Green (Green Reached)

State RED ⇒ Red → Green ✓

So, All states satisfy AFGREEN

Evaluate RED → AF GREEN

$(p \rightarrow q)$ means: $\neg p \vee q$

State	is RED?	AF GREEN true?	Formulae True
RED	Yes	Yes	✓
GREEN	NO	Yes	✓
YELLOW	NO	Yes	✓

So, property holds in all states ⇒ Since formulae holds in every state AG(RED → AF GREEN) is TRUE.

check safety property

$AG \neg (GREEN \wedge RED)$

check each state:-

- RED → only RED true → OK
- GREEN → only GREEN true → OK
- YELLOW → neither GREEN nor RED → OK

So Safety property also holds.

Write functional CTL properties

$AG((RED \wedge GO_GREEN) \rightarrow AX\ GREEN)$

$p \rightarrow q$

$\neg p \vee q$

Evaluate inside part

Firstly check

Case I: State = RED

This is true only if

AX GREEN me

from RED state :

SO, RE

Similarly do for Gre

Case II : State = GREEN

RED ∧ GO_

since RED =

Case III : State = YELLOW

Same logic, RED = fa

Since the inner formula is

holds.

Evaluate inside part

First check

Case I : State = RED RED ∧ GO_GREEN

This is true only in RED state, AND GO_GREEN = 1

AX GREEN means : For all next states, GREEN must hold

From RED state : If GO_GREEN = 1 ⇒ Next state = GREEN.

That matches FSM.

So, RED & GO_GREEN → next state is GREEN correct.

Similarly do for remaining two.

Case II : State = GREEN

RED ∧ GO_GREEN

Since RED = False, this becomes false.

False → AXGREEN

which is always True.

Case III : State = YELLOW

Same. Here, RED = False ⇒ So implication is automatically TRUE.

Since the inner formula is true in RED, GREEN, YELLOW ⇒ AG condition holds.